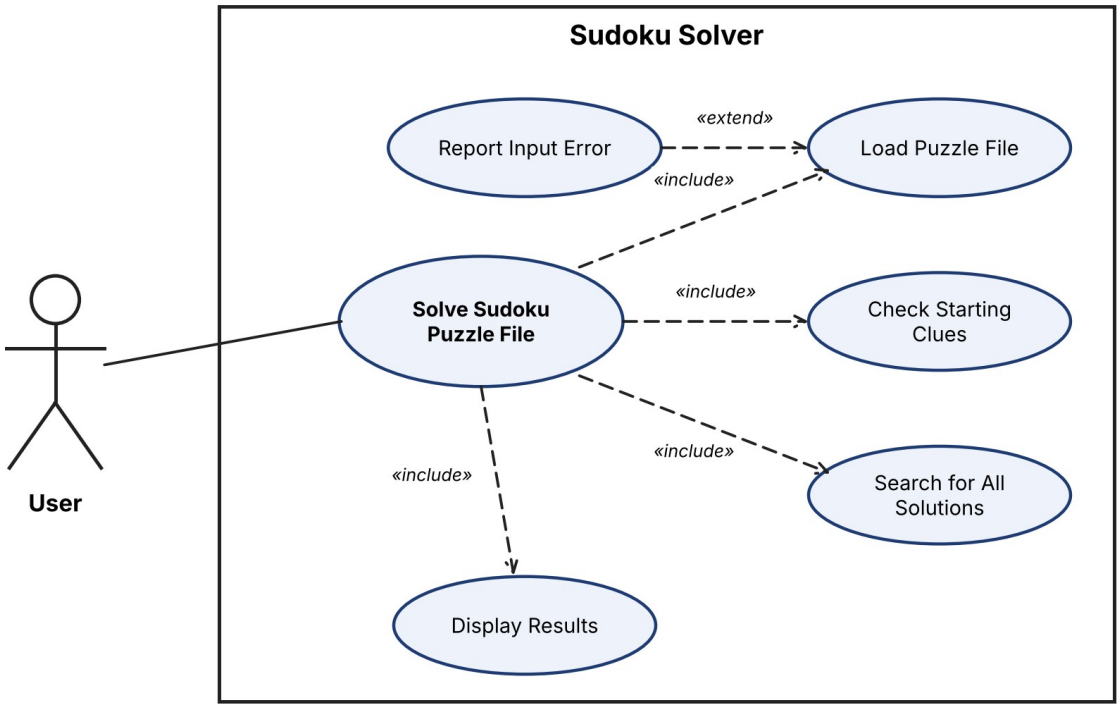
Use Case Model: Sudoku Solver

Diagram and use case text produced with help from Claude (Anthropic GenAI), based on the final version of sudoku_solver_revised.cpp.

Use Case Diagram



a. Identified Use Cases

The one actor is the User, who runs the program. The main use case, Solve Sudoku Puzzle File, is the goal the User wants done. Four smaller use cases are part of it every time (include), and one only happens when something goes wrong (extend).

Use case	Relationship	Purpose
Solve Sudoku Puzzle File	Main use case (User starts it)	Solve one puzzle file and report every solution.
Load Puzzle File	Included by Solve Sudoku Puzzle File	Read the 81 cells from the file and confirm each one is valid.
Check Starting Clues	Included by Solve Sudoku Puzzle File	Confirm the given digits do not already break a Sudoku

		rule.
Search for All Solutions	Included by Solve Sudoku Puzzle File	Use recursive brute force to find every solution.
Display Results	Included by Solve Sudoku Puzzle File	Show the file name, puzzle, and solutions or "No solution found".
Report Input Error	Extends Load Puzzle File	Explain what is wrong with a file that cannot be loaded.

b. Brief Description: Solve Sudoku Puzzle File

The User runs the program on one or more puzzle files. For each file, the system reads the puzzle, checks it, searches for every solution, and displays the file name, the original puzzle, and each solution found, or "No solution found" if there are none.

c. Step-by-Step Flow of Events

1. The User starts the program, either with no arguments (puzzle1.txt through puzzle5.txt are used) or with file names on the command line.
2. The system prints the name of the first puzzle file.
3. The system reads 81 cells from the file and confirms each is a digit 1-9 or an underscore.
4. The system prints the original puzzle.
5. The system checks that the starting digits do not repeat in any row, column, or 3x3 box.
6. The system finds the first empty cell and works out which digits could legally go there.
7. The system tries each candidate in turn: it places the digit and repeats steps 6-7 for the next empty cell. If a cell has no candidates, or a path fails, the system removes the digit and tries the next one (backtracking).
8. When no empty cells remain, the system confirms the board is a valid solution, prints it as a numbered solution, and keeps searching for others.
9. When the search is finished, the system prints the number of solutions found, or "No solution found" if there were none.
10. The system repeats steps 2-9 for the next file. After the last file, the program ends.

d. Detailed Description: Solve Sudoku Puzzle File

Use case name: Solve Sudoku Puzzle File

Primary actor: User

Goal: Obtain every valid solution to each puzzle file, or confirmation that none exists.

Trigger: The User runs the program.

Preconditions: The program is compiled. Each puzzle file exists in the working directory (or is named on the command line) and holds 81 whitespace-separated cells, written as digits 1-9 or underscores.

Postconditions (success): For every readable file, the console shows the file name, the puzzle, and either all solutions with a count or "No solution found". Puzzle files are not modified.

Postconditions (failure): A file that cannot be loaded produces an error message and no solving is attempted for it; the program continues with the next file.

Included use cases: Load Puzzle File; Check Starting Clues; Search for All Solutions; Display Results.

Extending use case: Report Input Error (extends Load Puzzle File).

Main flow

Steps 1-10 in section c.

Alternative flows

- **A1. Invalid or malformed file (at step 3):** The file is missing, ends early, has more than 81 cells, or has a cell that is not a single digit 1-9 or underscore (for example "x", "55", or "0"). The system displays "Error in <file>: <reason>", naming the row and column when a cell is at fault, skips the puzzle, and continues with the next file.
- **A2. Starting clues conflict (at step 5):** A digit is repeated in a row, column, or box. The system skips the search and displays "No solution found".
- **A3. No solution exists (at step 9):** Every candidate on every path fails. The system displays "No solution found".
- **A4. Multiple solutions (at step 8):** More than one board fills all cells. Each is printed and numbered, followed by the total count.
- **A5. Failed safety check (at step 8):** If a completed board fails the final rule check, the system prints an internal error message, does not display that board, and continues searching. This is not expected to occur.

Business rules: A digit may appear only once in each row, column, and 3x3 box. The search is exhaustive, so it never stops at the first solution.

Assumptions: The User runs the program from a console and reads the output there. Puzzle files use "_" for blank cells.